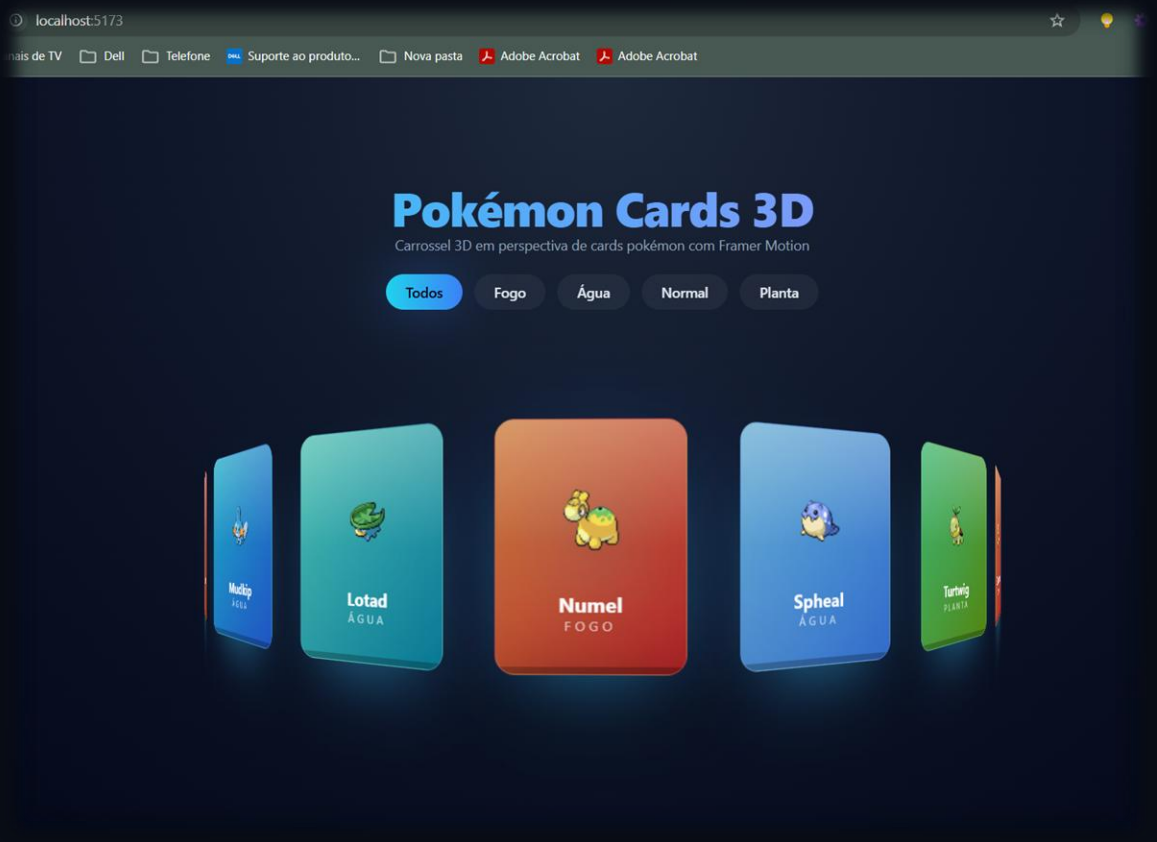
Trabalho #2 – Pacotes Extras

React Animations (Framer Motion)

Pokémon Cards 3D: Passo a Passo do Código

Maiara Torchelsen Saraiva



O **Framer Motion**

É a biblioteca de animação mais popular e poderosa para o **React**. Ela foi feita para tornar a criação de animações complexas, gestos (drag, hover, tap) e transições de páginas incrivelmente simples e declarativas.

Em vez de escrever dezenas de linhas de CSS ou controlar estados manuais com requestAnimationFrame, você define o comportamento desejado diretamente nas propriedades dos componentes.

1. O Conceito Fundamental: O componente motion

A base de tudo no Framer Motion é o objeto motion. Para animar qualquer elemento HTML padrão, você substitui a tag normal (div, button, h1) por sua versão ***motion (motion.div, motion.button, motion.h1***).





Funcionamento do carrossel

1. A lista de Pokémon é filtrada por tipo (Água, fogo....).

2. Cada carta recebe um ângulo (`passoAngulo * indice`) e é posicionada com `***rotateY***(ângulo) ***translateZ***(raio)`.

3. O grupo inteiro gira continuamente via `***useAnimationFrame***` e `***useMotionValue***`.

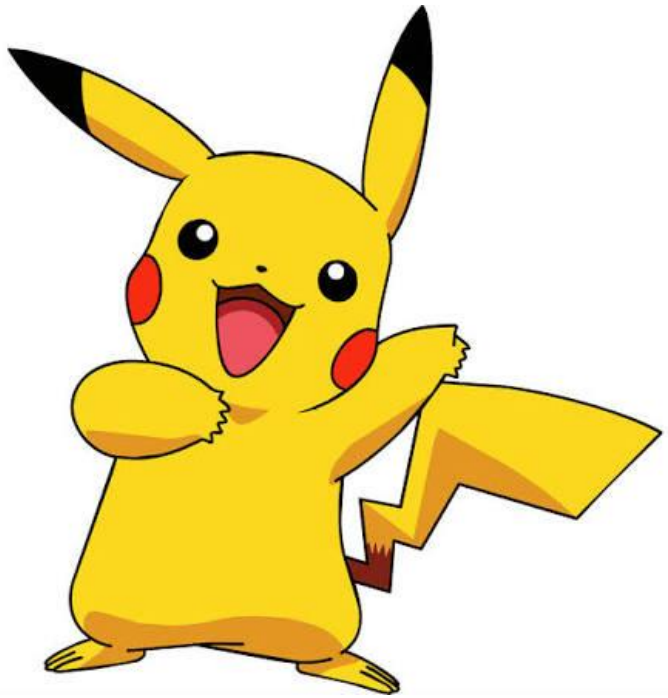
4. O raio cresce com a quantidade de cartas para evitar sobreposição, e uma escala de normalização compensa o efeito de perspectiva, mantendo o tamanho visual constante.

5. As cartas que passam de 90° ficam com a face traseira virada para o usuário e são escondidas com `*backface-visibility: hidden*`.

Dados dos Pokémon

Os sprites (**sprite** é uma imagem bidimensional (2D) que representa uma personagem) são carregados diretamente do repositório oficial de imagens da PokeAPI:

https://pokeapi.co/



1. CONFIGURAÇÃO INICIAL

Importações e Parâmetros 3D

- Importações do Framer Motion: Recursos essenciais para animações contínuas e controle de presença.
- Dimensões Estáticas: Cartas fixadas em 150x200px.
- Perspectiva do Palco: Valor fixo de 1200px define o ponto de fuga virtual.



```
1  import { useMemo, useRef, useState } from 'react';
2  import { motion, AnimatePresence, useMotionValue, useAnimationFrame } from 'framer-motion';
```

```
1  const LARGURA_CARTA = 150;
2  const ALTURA_CARTA = 200;
3  const PERSPECTIVA = 1200;
4  const ESCALA_FRENTE_ALVO = 1.3;
```

2. Dados

Lista de Pokémons

Cada item do array representa um Pokémon:

```
{
  id: 4,
  nome: 'Charmander',
  tipo: 'Fogo',
  cor: 'from-orange-400/80 to-red-600/80'
}
```

listaPokemons funciona como uma pequena base de dados local para mostrar os pokémons na interface das cartas e filtros.



```
const listaPokemons = [
  { id: 4, nome: 'Charmander', tipo: 'Fogo', cor: 'from-orange-400/80 to-red-600/80' },
  { id: 7, nome: 'Squirtle', tipo: 'Água', cor: 'from-cyan-400/80 to-blue-600/80' },
  { id: 37, nome: 'Vulpix', tipo: 'Fogo', cor: 'from-orange-300/80 to-rose-600/80' },
  { id: 54, nome: 'Psyduck', tipo: 'Água', cor: 'from-sky-300/80 to-blue-500/80' },
  { id: 58, nome: 'Growlithe', tipo: 'Fogo', cor: 'from-amber-400/80 to-red-500/80' },
  { id: 61, nome: 'Poliwag', tipo: 'Água', cor: 'from-cyan-300/80 to-indigo-500/80' },
  { id: 77, nome: 'Ponyta', tipo: 'Fogo', cor: 'from-yellow-400/80 to-orange-600/80' },
  { id: 90, nome: 'Shellder', tipo: 'Água', cor: 'from-teal-300/80 to-blue-600/80' },
```

3. ROTAÇÃO CONTÍNUA

useMotionValue + useAnimationFrame

Esses são dois recursos do **Framer Motion** usados juntos para criar a rotação contínua de 360°.

```
1  const rotacao = useMotionValue(0);
2   const pausado = useRef(false);
3
4   useAnimationFrame((_, intervalo) => {
5     if (!pausado.current) {
6       rotacao.set(rotacao.get() + (intervalo / 1000) * 10);
7     }
8   });
```

const rotacao = useMotionValue(0)
Cria um valor animado que começa em 0. Nesse caso,
ele representa o ângulo de rotação do carrossel.
Esse valor é aplicado ao elemento:
style={{ rotateY: rotacao }}
Quando rotação muda, o elemento gira no eixo Y.

useAnimationFrame
Executa uma função a cada quadro da animação. A cada quadro:
obtém o valor atual da rotação com *rotacao.get();*
calcula quanto tempo passou desde o quadro anterior;
adiciona uma pequena quantidade de graus;
atualiza a rotação com *rotacao.set*(...).

Os cards giram continuamente a **10 graus por segundo**, mas interrompe a atualização quando *pausado.current* estiver definido como true.

4. MATEMÁTICA ESPACIAL

Cálculo do Raio e Normalização

- **Ajuste de Circunferência: A fórmula trigonométrica com totalVagas expande o raio para não sobrepor cartas.**
- **Correção de Escala: O fator normalizacao encolhe o cilindro quando o raio cresce, mantendo a carta frontal com tamanho constante.**

Calcula dinamicamente o carrossel:
filtro selecionado
↓
lista de Pokémon exibida
↓
quantidade total de cartas
↓
ângulo entre as cartas
↓
raio do círculo
↓
correção da escala causada pela perspectiva
O resultado é que as cartas ficam distribuídas em um círculo 3D, com espaçamento adaptável, no mínimo 8 posições e uma carta frontal sem aumento exagerado pela perspectiva.

```
export default function GaleriaPokemon() {
  const [filtro, setFiltro] = useState('Todos');

  const cartas = useMemo(() => {
    const base = listaPokemons.filter((pokemon) => filtro === 'Todos' || pokemon.tipo === filtro);
    const origem = base.length ? base : listaPokemons;

    const minimoVagas = 8;
    const repeticoes = Math.max(1, Math.ceil(minimoVagas / origem.length));
    const vagas: typeof listaPokemons = [];
    for (let volta = 0; volta < repeticoes; volta++) vagas.push(...origem);
    return vagas;
  }, [filtro]);

  const totalVagas = cartas.length;
  const passoAngulo = 360 / totalVagas;

  // O raio cresce para acomodar mais cartas sem sobreposição.
  const raio = Math.min(
    Math.max(240, 210 / (2 * Math.sin(Math.PI / totalVagas))),
    PERSPECTIVA * 0.82
  );

  // Normalização: mantém o tamanho da carta frontal constante
  const escalaFrente = PERSPECTIVA / (PERSPECTIVA - raio);
  const normalizacao = Math.min(1.1, ESCALA_FRENTE_ALVO / escalaFrente);
```

O raio representa a distância entre o centro do carrossel e cada carta.
As cartas são posicionadas usando:

transform: `rotateY(${angulo}deg) translateZ(${raio}px)`

Ou seja:

1. **a carta gira ao redor do eixo Y;**
2. **depois se afasta do centro pela distância definida em raio.**

5. DISTRIBUIÇÃO E TRANSIÇÕES

Posicionamento Espacial e Filtros

```jsx
<AnimatePresence>
                  {cartas.map((pokemon, indice) => {
                    const angulo = passoAngulo * indice;

                    return (
                      <motion.div
                        key={`${pokemon.id}-${indice}`}
                        className="absolute inset-0 [transform-style:preserve-3d]"
                        initial={{ opacity: 0 }}
                        animate={{ opacity: 1 }}
                        exit={{ opacity: 0 }}
                        transition={{ duration: 0.35 }}
                        style={{
                          transform: `rotateY(${angulo}deg) translateZ(${raio}px)`,
                          backfaceVisibility: 'hidden',
                        }}
                      >
```

Ao alternar o filtro de tipos (Fogo, Água, etc.), as cartas entram e saem da lista. O **AnimatePresence** gerencia a transição de desmontagem das cartas antigas e montagem das novas

A classe Tailwind [transform-style:preserve-3d] mantém a profundidade das cartas filhas.

Posiciona a carta no espaço 3D:
rotateY gira a carta ao redor do eixo vertical;
translateZ afasta a carta do centro, formando o círculo;
backfaceVisibility: 'hidden' esconde o verso da carta quando ela estiver virada para trás.

O código cria cada carta, distribui as cartas ao redor de um círculo 3D e anima sua entrada e saída.

6. MICROINTERAÇÕES

Animações de entrada e de destaque ao passar o mouse.

Microinterações com whileHover e Animações em Mola (spring)

```jsx
<motion.div
                    className={`relative flex h-full w-full flex-col items-center justify-center gap-2 overflow-hidden rounded-2xl border border-white/15 bg-gradient-to-br ${pokemon.cor} p-4 text-center shadow-[0_30px_60px_-25px_rgba(56,189,248,0.55)] backdrop-blur`}
                    style={{ backfaceVisibility: 'hidden' }}
                    initial={{ scale: 0.6 }}
                    animate={{ scale: 1 }}
                    whileHover={{ scale: 1.08, y: -8 }}
                    transition={{ type: 'spring', stiffness: 260, damping: 20 }}
                  >
```

```jsx
<motion.img
                      src={`https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/${pokemon.id}.png`}
                      alt={pokemon.nome}
                      className="relative z-10 h-24 w-24 drop-shadow-[0_20px_35px_rgba(14,165,233,0.45)]"
                      whileHover={{ rotate: [0, -8, 8, -6, 6, 0], scale: 1.12 }}
                      transition={{ duration: 0.8 }}
                    />
```

Nas cartas: Elevação e expansão suave via física de mola (type: 'spring', stiffness: 260, damping: 20).

Nas imagens dos Pokémon: Um efeito de balanço usando uma sequência de valores (*keyframes*) no rotate:
whileHover --o array rotate: [0, -8, 8, -6, 6, 0]

Resumo do Código

- Carrossel 3D em anel: as cartas são posicionadas em círculo com `rotateY` + `translateZ`, criando profundidade e perspectiva reais.

- Normalização de perspectiva: o anel é reescalado automaticamente para que a carta da frente mantenha sempre o mesmo tamanho na tela, mesmo adicionando mais Pokémon.

- Sem textos espelhados: `backface-visibility` oculta as cartas que viram de costas, evitando nomes e imagens invertidos.

- Filtros por tipo: `Todos`, `Fogo`, `Água`, `Normal` e `Planta`. Os filtros com poucas cartas são repetidos para manter o anel cheio e sem espaços vazios.

- Interação: a rotação pausa ao passar o mouse e as cartas dão zoom/inclinação no hover.

- Layout compacto e responsivo: conteúdo centralizado, cartas próximas aos filtros.

"Combinando a matemática do 3D com a performance do Framer Motion para criar um carrossel fluido e leve no React."